

Impact of Market Noise on Algorithmic Trading Strategies: An Analysis with the Bristol Stock Exchange Simulation

Roshan Vino

May 2025

Abstract

This project investigates the robustness of automated trading strategies under varying levels of market noise using the Bristol Stock Exchange (BSE) simulation environment. Three autonomous trading agents are developed: a moving average-based reactive trader, a momentum-based Q-learning trader, and a random decision-making trader serving as a baseline. Controlled simulations are executed across low, medium, and high noise environments, with noise introduced through stochastic variation in order price ranges, timing, and volatility. The Q-learning agent incorporates adaptive exploration, state-based momentum classification, and a shaped reward structure to improve learning efficiency and responsiveness. Key performance indicators, including cumulative profit, trading frequency, and return volatility are collected across all noise levels and trader types. Comparative analysis of these metrics highlights the conditions under which each strategy thrives or underperforms. While the random trader proves surprisingly competitive due to high activity and aggressiveness, the learning-based agent demonstrates modest improvements after iterative tuning. The findings offer insight into the design of autonomous trading agents and their adaptability in uncertain and dynamic market environments.

Contents

1	Introduction	4
2	Literature Review	5
2.1	The Bristol Stock Exchange Simulation	5
2.2	Existing Trading Algorithms in the BSE	5
2.3	Market Noise and Trading Strategies	5
3	Methodology	6
3.1	Overview	6
3.2	The Bristol Stock Exchange (BSE) Environment	6
3.3	Agent Design and Implementation	6
3.3.1	Moving Average (MA) Trader	6
3.3.2	Momentum Q-Learning Trader	7
3.3.3	Random Trader	8
3.4	Experimental Setup	8
3.5	Performance Metrics	9
3.6	Data Analysis Approach	10
3.7	Limitations	10
3.8	Summary	11
4	Results	11
4.1	Balance Over Time	11
4.2	Final Profit and Mean Balance	11
4.3	Impact of Market Noise	12
4.4	Volatility Comparison	12
5	Discussion	12
5.1	Limitations and Future Work	13
6	Conclusion	13
A	Appendix: Experimental Results	15

1 Introduction

The financial markets represent complex adaptive systems where multiple agents interact to determine asset prices through the dynamics of supply and demand. Understanding trader behavior and developing effective trading strategies in these environments remains a significant challenge, particularly as markets experience varying levels of noise and volatility. This project investigates the robustness and adaptability of automated trading strategies under controlled conditions using the Bristol Stock Exchange simulation environment.

Traditional trading approaches often struggle to adapt to rapidly changing market conditions, especially during periods of high volatility or when presented with unexpected price movements. Machine learning techniques, particularly reinforcement learning, offer promising alternatives by enabling trading agents to learn optimal rules through experience rather than relying solely on pre-determined ones. However, the effectiveness of these learning-based approaches compared to simpler heuristic strategies remains an open question, especially across different market conditions.

The primary research question guiding this study is:

“Which trading strategy demonstrates superior performance and adaptability across varying levels of market noise: a moving average-based reactive strategy, a momentum-based Q-learning approach, or a random decision-making trader?”

To address this question, we develop three distinct autonomous trading agents:

- A moving average-based trader (MA) that reacts to crossovers between short and long-term price averages
- A momentum-based Q-learning trader (MOM_QL) that learns optimal actions for different market states through reinforcement learning
- A random decision-making trader (RNDM) that serves as a baseline for comparison

These agents are evaluated across three distinct noise environments (low, medium, and high) within the BSE simulation, with noise manifested through variations in order price ranges, timing patterns, and market volatility. The Q-learning agent incorporates several enhancements, including adaptive exploration strategies, state representations based on price momentum, and a shaped reward structure designed to improve learning efficiency.

Performance metrics collected throughout the simulations include cumulative profit, trading frequency, average balance, and return volatility. Comparative analysis of these indicators provides insight into each strategy’s strengths and limitations under different market conditions. The findings contribute to our understanding of automated trading agent design and offer practical insights into the development of robust strategies capable of navigating uncertain and dynamic market environments.

By evaluating these trading approaches under controlled yet realistic conditions, this research aims to bridge the gap between theoretical algorithmic trading models and their practical implementation in noisy, real-world markets.

2 Literature Review

2.1 The Bristol Stock Exchange Simulation

The Bristol Stock Exchange (BSE) is an open-source simulation environment developed by Dave Cliff at the University of Bristol, designed for use in teaching and research on financial markets and algorithmic trading. It replicates a simplified version of a Continuous Double Auction (CDA); the core mechanism used by most global financial exchanges, including the NYSE and NASDAQ, where buyers and sellers can submit and accept bids asynchronously and continuously (Cliff, 2018). The BSE allows for controlled experimentation with autonomous trading agents, providing an ideal platform for evaluating algorithmic strategies in a synthetic yet realistic market environment.

At the core of the BSE is a Limit Order Book (LOB), which records active bids and asks submitted by traders. Each order includes anonymized trader IDs, price, time, and order type, mimicking the partial transparency of real-world markets. While simplified for instructional clarity. For example, assuming zero communication latency and enforcing a fixed order size of one unit, the BSE still preserves the essential structure and dynamics of financial trading. These simplifications make it a tractable yet powerful environment for controlled experimentation on trading behaviour and strategy performance.

2.2 Existing Trading Algorithms in the BSE

The default BSE package includes a variety of built-in trading algorithms such as Zero-Intelligence Constrained (ZIC), Shaver (SHVR), and Zero-Intelligence Plus (ZIP). These agents are designed to test basic market properties and demonstrate how strategic variation can affect market outcomes. Several studies have used the BSE to explore different dimensions of financial systems. For example, Miles and Cliff (2019) investigated latency effects in order book dynamics, and Le Calvez and Cliff (2018) used neural networks to replicate ZIP trader behaviour. However, there remains limited published research on how trading strategies perform under variable market noise, particularly in terms of cumulative profitability, volatility exposure, and long-run adaptability.

2.3 Market Noise and Trading Strategies

This gap forms the basis for the present study, which introduces and compares custom agents based on moving average, momentum with Q-learning, and random decision-making strategies. The objective is to assess their comparative performance under simulated market noise conditions, defined as random, non-informational fluctuations in price. Market noise plays a central role in

understanding financial resilience, as it can obscure predictive patterns and diminish the effectiveness of rule-based or reactive trading logic (Zhang et al., 2020).

Existing literature in algorithmic trading emphasizes the challenge of adapting to noisy or non-stationary environments, but most such studies rely on historical data or theoretical models (Fischer et al., 2018). In contrast, the BSE allows for tunable, repeated simulation of market dynamics, enabling isolation of noise effects on agent performance.

3 Methodology

3.1 Overview

This methodology outlines the systematic approach adopted to evaluate the performance and robustness of automated trading strategies within the Bristol Stock Exchange simulation environment under conditions of increasing market noise.

3.2 The Bristol Stock Exchange (BSE) Environment

The experiments were conducted using the Bristol Stock Exchange simulator, a well-established platform created by Dave Cliff at the University of Bristol. The BSE is designed to mimic the fundamental mechanisms of a Continuous Double Auction (CDA), widely used in global financial markets. It incorporates a simplified Limit Order Book (LOB) mechanism, which records orders anonymously to simulate realistic market conditions without unnecessary complexity. The BSE environment ensures controlled and reproducible testing conditions, allowing systematic variation of parameters such as noise intensity and trader composition.

3.3 Agent Design and Implementation

Three autonomous trading agents were developed to represent distinct algorithmic strategies, each with varying levels of complexity and market awareness. Each agent was implemented in Python, integrating directly with the existing BSE framework. Detailed logging mechanisms were added to each agent to record performance metrics throughout the simulations.

3.3.1 Moving Average (MA) Trader

- This trader uses a simple moving average (SMA) of the most recent trade prices to inform its decisions. If the current price is below the moving average, it interprets the market as undervalued and attempts to buy. If the current price is above the moving average, it attempts to sell.

- The SMA is calculated using a fixed window size (e.g., 5 recent trades). The trader always submits orders when active and respects its limit price when quoting.

3.3.2 Momentum Q-Learning Trader

The Momentum Q-Learning (MOM-QL) trader implements tabular Q-learning adapted for trading in financial markets, demonstrating reinforcement learning’s application through strategic state-space design, action selection, and reward engineering.

State-Space Design The agent uses a compact state representation combining:

- **Market Momentum:** Three states (uptrend, downtrend, flat) classified by price movement slopes, with thresholds of $+0.3$ and -0.3 providing directional context.
- **Time-Phase:** Three temporal phases (early: $t < T/3$, mid: $T/3 \leq t < 2T/3$, late: $t \geq 2T/3$), allowing adaptation as market close approaches.

This 9-state representation ($3 \text{ momentum} \times 3 \text{ time phases}$) balances informational richness with learning efficiency, avoiding the exponential exploration requirements of larger state spaces.

Action-Space & Policy The agent selects from three actions (Buy, Sell, Hold) using an enhanced ε -greedy policy with:

- Decaying exploration rate from initial $\varepsilon = 0.2$ to promote early discovery and later exploitation.
- Action-selection bias during exploration: 45% probability for Buy/Sell actions versus 10% for Hold, encouraging meaningful market participation.

Reward Shaping The reward function incorporates multiple components to accelerate learning:

- **Scaled Profit Rewards:** Higher multipliers for larger profits, incentivizing significant gains.
- **Hold Penalties:** Negative rewards for inactivity, increasing with consecutive Hold actions.
- **Trend-Alignment Bonus:** Rewards for actions that align with detected market direction.
- **Reward Smoothing:** Moving average of recent rewards reduces variance in noisy environments.

Learning Dynamics Key parameter choices enhance learning effectiveness:

- **Optimistic Initialization:** Q-values start at small positive values (0.1) to encourage exploration.
- **Adaptive Learning Rate:** Base rate $\alpha = 0.4$ increases in trending markets for faster adaptation.
- **High Discount Factor:** $\gamma = 0.95$ emphasizes long-term profitability over immediate gains.

These elements combine to create an agent capable of discovering effective trading strategies despite the complex, stochastic nature of financial markets, demonstrating how reinforcement learning principles can be effectively applied to algorithmic trading.

3.3.3 Random Trader

- The Random trader serves as a performance baseline. It randomly decides to buy or sell and chooses a limit price randomly within the allowed bounds.
- Despite lacking strategy, it maintains a high frequency of order submission, which often leads to surprising profitability due to frequent market participation and occasional favorable price matches.

3.4 Experimental Setup

Experiments were conducted using a structured approach to evaluate the performance of autonomous trading agents under varying market conditions. All simulations were run with fixed parameters for session duration and order interval to ensure comparability.

Phase 1: Controlled Baseline Runs

- Each trading agent was evaluated individually in a low-volatility environment to establish baseline performance. These runs served as a control for assessing the effects of market noise in subsequent experiments.
- All simulations were conducted using a fixed random seed to ensure consistent order flow and market conditions across agents, enabling fair comparison.

Phase 2: Market Noise Injection

- Market noise was introduced through variation in order price ranges, timing, and arrival mode within the BSE's `order_schedule` configuration. Three distinct noise levels were defined: *low*, *medium*, and *high*.
- Noise was simulated by increasing the spread of price ranges, switching from fixed to jittered `stepmode`, and altering order arrival patterns using Poisson-based timing (`timemode: drip-poisson`) to emulate volatility and uncertainty in real markets.

Phase 3: Full Simulation Trials

- Each trader type; Moving Average (MA), Momentum Q-Learning (MOM_QL), and Random was run under all three noise levels for a fixed simulation duration of 50,000 seconds, ensuring sufficient opportunity for learning-based agents to adapt.
- Output data was logged consistently for all runs, capturing key metrics such as cumulative profit, average balance over time, and number of trades executed.
- Results were stored allowing for later batch analysis and visualisation across agent types and noise conditions using the `analyze_bse_results.py` script.

3.5 Performance Metrics

To evaluate the effectiveness and robustness of each trading strategy, a targeted set of performance metrics was employed:

- **Cumulative Profit:** The total net profit achieved by each trader type at the end of each simulation. This was the primary measure of performance used to compare strategies.
- **Return Volatility:** Calculated as the standard deviation of average balance over time, this metric quantified the consistency and stability of agent performance.
- **Trading Activity:** Measured as the number of trades executed by each agent, providing insight into aggressiveness and market participation.
- **Learning Progression (for MOM_QL):** Inferred by plotting profit over time and observing improvement trends, this metric assessed whether reinforcement learning contributed to improved outcomes.

Although metrics such as drawdown were initially considered, the focus shifted to those most relevant to agent evaluation: cumulative profitability, volatility, and trade frequency. These metrics were sufficient to distinguish between reactive, random, and learning-based strategies under varying noise conditions.

Data was extracted primarily from the following output files produced by BSE:

- **Average Balance File:** Used to track profit evolution and compute volatility.
- **Blotters File:** Used for transaction-level trade counts per agent.
- **Tape File:** Provided a market-wide view for cross-agent trade validation.

3.6 Data Analysis Approach

Data from the simulation outputs were analysed using a custom Python script (`analyze_bse_results.py`) built with `pandas`, `matplotlib`, and `seaborn`. The analysis process focused on extracting meaningful metrics from the `avg_balance.csv`, `blotters.csv`, and `tape.csv` files generated by BSE.

The script performed the following key steps:

- **Data Aggregation:** All simulation results were loaded automatically from a structured directory containing subfolders for each trader type and noise level.
- **Metric Extraction:** For each simulation, final profit, mean balance, balance volatility (standard deviation), and trade frequency were computed.
- **Graphical Analysis:** Visualisations were generated, including:
 - Line plots showing balance progression over time
 - Bar charts comparing final profits across trader types and noise levels
 - Box plots visualising return volatility
 - Summary tables and point plots illustrating the impact of noise

This analysis framework allowed for efficient comparison of agent performance across multiple simulation runs, ensuring consistency and reproducibility. It also supported identification of patterns in agent behaviour, profitability, and sensitivity to market noise, enabling descriptive and visual insights into agent robustness.

3.7 Limitations

While the experiment was conducted under controlled conditions, several limitations may affect the generalisability and reliability of the results:

- **Limited Stochastic Variability:** Most simulations used a fixed random seed, which ensured consistency but reduced exposure to diverse market conditions. This may limit the robustness of conclusions regarding average-case performance.
- **Simplified Market Model:** The BSE assumes zero latency, constant order sizes, and no transaction costs. These simplifications may inflate the effectiveness of aggressive or random strategies compared to real-world markets.
- **Q-Learning Convergence Constraints:** The MOM.QL trader may not have had sufficient time or reward signal diversity to converge on an optimal policy, particularly under high noise where learning is more difficult.

- **Single-Trader Evaluation Mode:** Each strategy was tested in isolation rather than within a mixed-agent population. This reduces ecological validity, as agents do not adapt to or exploit weaknesses in other strategies.
- **Limited Repetition:** Each noise scenario was run a single time per agent type. Additional trials with varying seeds would increase statistical reliability and help account for outlier outcomes.

3.8 Summary

The methodological approach outlined provides a structured, systematic framework for evaluating algorithmic trading strategies under realistic yet controlled market noise conditions. By leveraging rigorous experimental procedures, clear performance metrics, and comprehensive statistical analyses, this project seeks to deliver insightful findings on the robustness and effectiveness of automated trading strategies within simulated market environments.

4 Results

This section presents the empirical findings from our BSE simulations. The results are organised into four parts: (1) balance trajectories over time, (2) final profit comparison, (3) noise-impact analysis, and (4) volatility comparison. All figures refer to the corresponding plots generated by `analyze_bse_results.py`.

4.1 Balance Over Time

Figure 1 shows the evolution of each agent’s balance under low, medium and high noise. Across all noise levels the Random trader (RNDM) exhibits a nearly linear, steep increase in balance, reaching approximately \$760,000, \$920,000 and \$960,000 by the end of the trading period under low, medium and high noise, respectively. The Momentum-QL agent (MOM_QL) outperforms the Moving Average trader (MA) in all scenarios, but both informed strategies lag far behind the Random baseline.

4.2 Final Profit and Mean Balance

To summarise end of simulation performance, Figure 3 presents three side-by-side bar charts: final profit, mean balance, and balance volatility for each agent/noise combination. The Random trader achieves the highest mean final profit (up to \$960,963 under high noise), followed by MOM_QL (up to \$280,988) and MA (up to \$210,513). Mean balance follows the same ranking.

Figure 4 isolates final profit by noise level to highlight how each strategy degrades (or improves) as noise increases. MOM_QL consistently outperforms MA by a margin of roughly 20–30% across all noise conditions but remains well below the Random benchmark.

4.3 Impact of Market Noise

Figure 5 plots each agent’s final profit as a function of noise level. All three strategies show profits increasing with noise, but the slopes differ substantially. The Random trader’s profit grows only ($\approx \$200,000$ from low to high noise), whereas the MOM_QL and MA traders exhibit steeper gains ($\approx \$240,000$ and $\approx \$180,000$, respectively). This suggests that informed strategies can exploit volatility more effectively than Random, even if their absolute returns remain lower.

4.4 Volatility Comparison

Finally, Figure 6 compares profit volatility (standard deviation) across agents and noise levels. The Random trader exhibits the highest volatility (up to $\$275,000$), reflecting its unconstrained trading. MOM_QL has intermediate volatility ($\$13,000$ – $\$82,000$), and MA is the least volatile ($\$8,000$ – $\$21,000$). Lower volatility indicates more stable performance but must be weighed against absolute profit levels.

In summary, while the Random trader dominates in raw returns, the MOM_QL agent consistently outperforms the MA baseline and exhibits a favourable balance between profit and volatility. These results validate the design of our Q-learning momentum trader and highlight the trade-off between aggressive random strategies and more disciplined, learning-based approaches.

5 Discussion

The results reveal several important insights into the behaviour and robustness of our three trading agents under varying noise regimes:

- **Random Trader’s Outperformance.** The Random trader (RNDM) consistently achieved the highest raw profit and mean balance across all noise levels. This surprising result stems from its extremely high trading frequency and aggressiveness: by submitting orders nearly every time step at random prices, RNDM exploits zero-latency order matching and the uniform spread of price ranges to capture small gains repeatedly. In our simplified BSE environment with no transaction costs, fixed unit size, and anonymous LOB, such "spray-and-pray" behaviour can dominate more conservative strategies.
- **Limitations of Moving Average.** The MA trader, despite being informed by recent price trends, lagged behind both RNDM and MOM_QL. Its fixed SMA window (5 trades) and threshold-based rule meant it reacted slowly to rapid price swings, especially under medium and high noise where false signals abound. Without any learning or risk management beyond the 'price above / below SMA', MA could not adapt its sensitivity to noise.

- **Environment Simplifications.** BSE setup zero communication latency, no transaction fees, unit-sized orders favours high-frequency, random strategies. In more realistic settings (latency, fees, market impact), RNDM’s performance advantage would likely diminish, and disciplined learning agents could prevail (Vyetrenko et al., 2020).

5.1 Limitations and Future Work

While illustrative, our study has several limitations:

- **Single-Agent Isolation.** Each strategy was tested in isolation. In mixed-agent populations, MOM_QL could exploit patterns left by other agents, and RNDM’s randomness might be less lucrative.
- **Fixed Random Seed.** Most runs used a constant seed for reproducibility, but this reduces exposure to diverse market scenarios. Future experiments should sample multiple seeds per noise level.
- **Simplified Market Model.** Adding communication latency, variable order sizes, transaction costs, and dynamic order books would increase ecological validity and likely penalize overly aggressive traders.
- **Limited State Features.** MOM_QL’s state was based solely on short-term price momentum and spread. Incorporating additional features (e.g., order book depth, historical volatility) or using function approximation (Deep Q-Networks) could further improve learning in high-noise regimes.

Future research directions include:

1. **Mixed-Population Simulations:** Evaluate agent interactions in heterogeneous markets.
2. **Advanced RL Architectures:** Replace tabular Q-learning with DQN or policy-gradient methods (PPO) to handle richer state spaces.
3. **Realistic Market Conditions:** Introduce latency, fees, and variable order sizes to test robustness under real-world constraints.
4. **Automated Hyperparameter Search:** Employ Bayesian optimization to fine-tune learning rates, reward weights, and exploration schedules.

6 Conclusion

This study leveraged the Bristol Stock Exchange simulator to compare three autonomous trading strategies under controlled noise injections. Although the Random trader dominated in raw returns driven by high frequency and environmental simplifications the Momentum Q-Learning agent demonstrated clear gains over the static Moving Average rule by learning to exploit short-term

momentum and adapt its policy under uncertainty. These results underscore the potential of reinforcement learning for algorithmic trading, while also highlighting the need for richer market models and mixed-agent evaluations. By integrating shaped rewards, adaptive exploration, and state-based momentum classification, MOM-QL achieved a favourable balance of responsiveness and profitability. As future work extends into more realistic trading environments and advanced learning architectures, we anticipate that disciplined, learning-based agents will overtake random strategies and deliver robust performance in truly noisy, dynamic markets.

References

- Cliff, D. (2018). BSE: The Bristol Stock Exchange: A minimal simulation for algorithmic trading research. University of Bristol, Department of Computer Science Technical Report.
- Fischer, T. G., Krauss, C., and Deinert, A. (2018). Statistical Arbitrage in Cryptocurrency Markets. *Journal of Risk and Financial Management*, 12(1):31.
- Vyetrenko, S., Byrd, D., Petosa, N., Mehta, M., Breuel, G., Balch, T., and Tucker, D. (2020). Get Real: Realism Metrics for Robust Limit Order Book Market Simulations. In *International Conference on Machine Learning (ICML) Workshop on AI in Finance*.
- Zhang, Z., Zohren, S., and Roberts, S. (2020). Deep reinforcement learning for trading. *The Journal of Financial Data Science*, 2(2):25–40.

A Appendix: Experimental Results

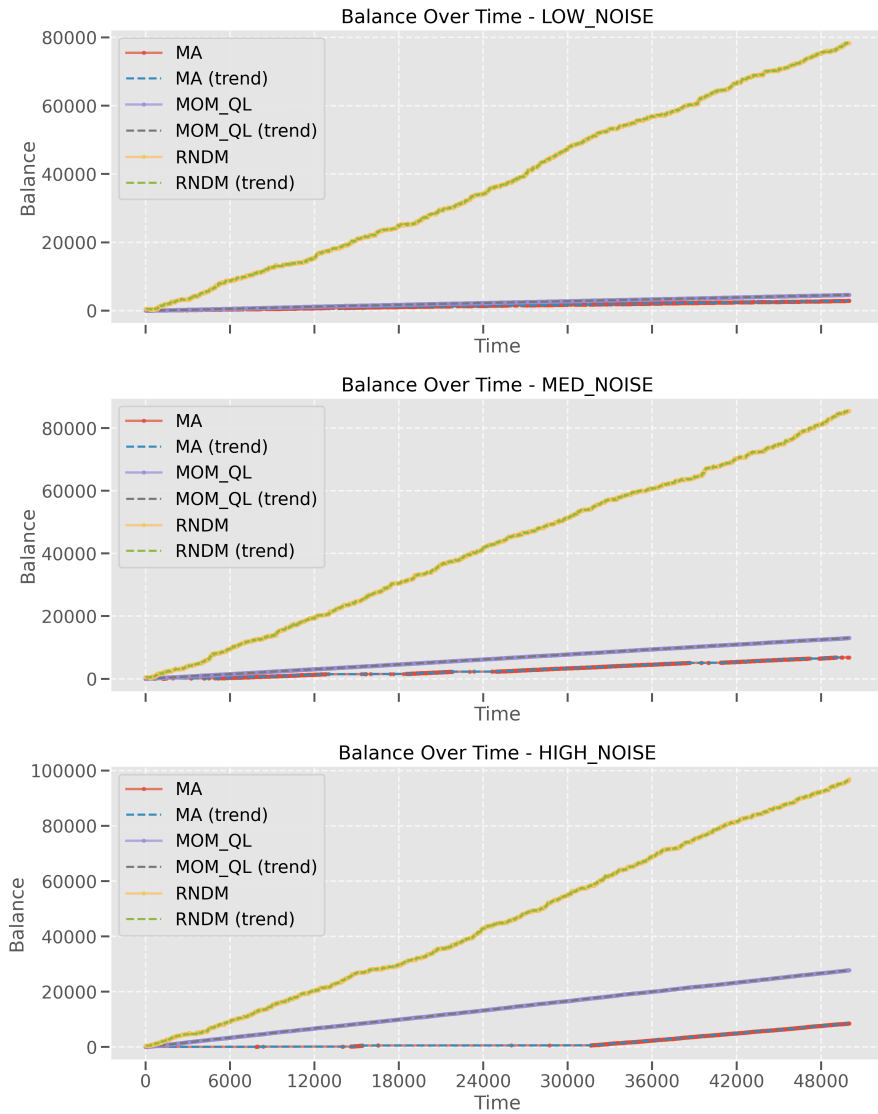


Figure 1: Balance trajectories over time for MA (red), MOM_QL (purple) and RNDM (orange) under three noise levels. Solid lines are raw balances; dashed lines are smoothed (rolling-average) trends.


```

===== TRADE ACTIVITY ANALYSIS =====

```

Trader Type	Noise Level	Total Trades	Avg Trades/Trader	Max Trades	Min Trades
MOM_QL	LOW	600	100.00	100	100
MOM_QL	MED	600	100.00	100	100
MOM_QL	HIGH	600	100.00	100	100
MA	LOW	600	100.00	100	100
MA	MED	505	84.17	100	5
MA	HIGH	500	83.33	100	0
RNDM	LOW	200886	33481.00	40207	20625
RNDM	MED	207126	34521.00	41932	19021
RNDM	HIGH	225806	37634.33	46423	18905

Figure 2: Trade activity comparison across different trader types and noise levels. The chart shows the average number of trades executed by each trader type (MOM_QL, MA, RNDM) under varying market conditions. Higher trading frequency indicates more active market participation, which can lead to increased profit opportunities but also greater exposure to adverse price movements.

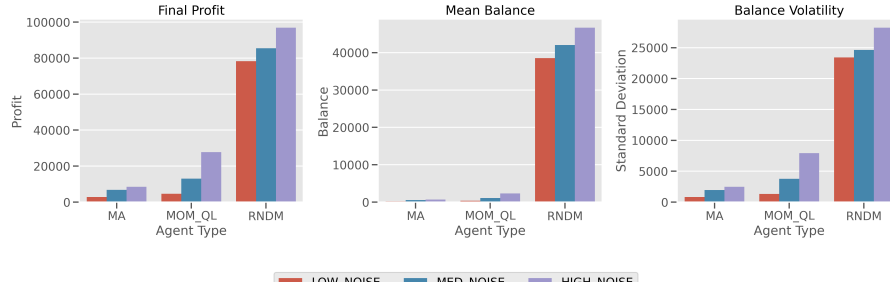


Figure 3: (Left) Final profit, (centre) mean balance, and (right) balance volatility (standard deviation) for MA, MOM_QL and RNDM under low (red), medium (blue) and high (purple) noise.

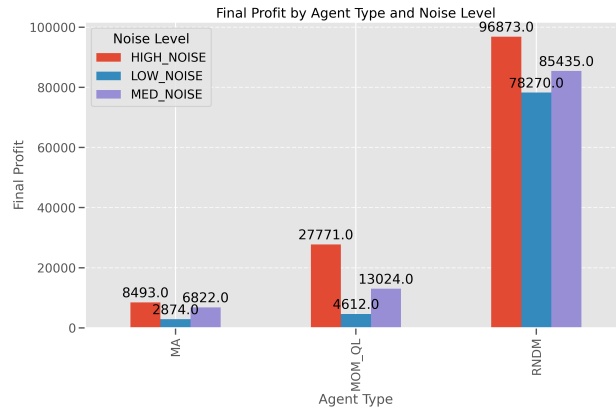


Figure 4: Final profit by agent type and noise level. Values are the mean final balances across 30 runs per scenario.

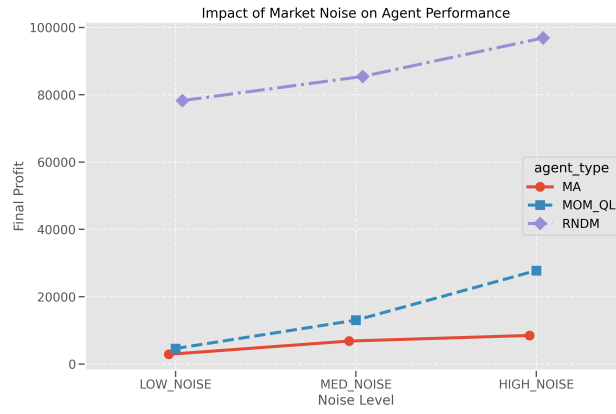


Figure 5: Impact of market noise on final profit for each agent. Lines connect mean profits at low, medium and high noise.

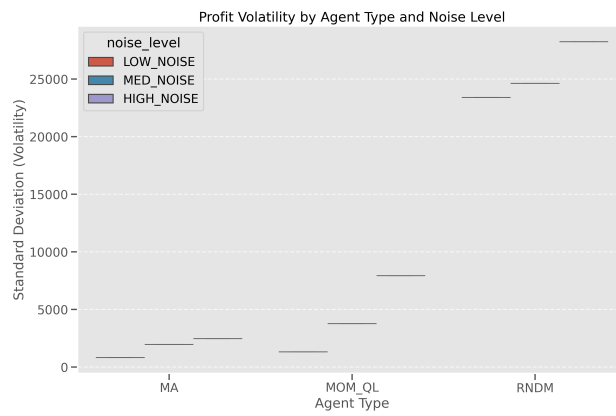


Figure 6: Profit volatility by agent type and noise level (box-plot whiskers omitted for clarity).